

Student academic portal

II B.Tech I Semester

**OBJECT ORIENTED PROGRAMMING THROUGH JAVA
LAB PROJECT**

Submitted By

25211A0577

25211A0578

25211A05B1

25211A05D0



Under the kind guidance of

Dr. T. Subba Reddy

Department of Computer Science and Engineering

B V Raju Institute of Technology

(UGC Autonomous, Accredited by NBA and NAAC)

Vishnupur, Narsapur, Medak (Dist), Telangana State, India - 502313.

Table of Contents:

S.No	Topic Name	Page No.
1	INTRODUCTION	1
2	MODULE DESCRIPTION	1-6
3	IMPLEMENTATION	7-16
4	OVERALL PROGRAM ALGORITHM	17
5	RESULTS	18
6	CONCULSION	19

INTRODUCTION

- Background of the project: The project is developed to provide a simple and efficient solution for managing information and performing required operations using a computer-based system. It helps reduce manual work and makes the process faster, more organized, and easier to use.
- Problem being addressed: Traditional manual methods can be time-consuming and may lead to errors, data duplication, or difficulty in maintaining and retrieving information. Managing large amounts of data manually can also reduce efficiency.
- Overview of the solution: The proposed system provides a computerized solution to manage the required data efficiently. It allows users to perform necessary operations such as adding, updating, searching, deleting, and displaying information. The system improves accuracy, saves time, and provides an organized way to handle data.

STUDENT ACADEMIC PORTAL — MODULE DESCRIPTION

MODULE 1: STUDENT MANAGEMENT

The **Student Management Module** is responsible for storing and managing student information in the Student Academic Portal. It maintains details such as **Roll Number, Student Name, Phone Number, Email, Branch, Year, and Password**.

Student information is stored in the **MySQL database**, where the Roll Number is used as the unique identifier for each student.

This module allows the administrator to:

- Add new students
- Search students using Roll Number
- Update student information
- Delete student records
- View all registered students
- Authenticate students during login

The module is implemented using the `Student` class and `StudentDAO` class.

Main functions: `addStudent()`, `findStudent()`, `updateStudent()`, `deleteStudent()`, `getAllStudents()`, and `loginStudent()`.

Java concepts used: Classes, Objects, Inheritance, Encapsulation, JDBC, MySQL, PreparedStatement, and CRUD operations.

MODULE 2: ADMIN MANAGEMENT

The **Admin Management Module** manages administrator authentication and administrator information in the Student Academic Portal.

The administrator is responsible for managing student records, academic results, and attendance information.

Admin details include:

- Admin ID
- Username
- Name
- Phone Number
- Email
- Password

The admin credentials are stored in the **MySQL database**. The module verifies the username and password during login and provides access to the Admin Dashboard. The module is implemented using the `Admin` and `AdminDAO` classes.

Main functions: `login()` and `displayDetails()`.

Java concepts used: Classes, Objects, Inheritance, Encapsulation, JDBC, MySQL, and Authentication.

MODULE 3: ACADEMIC RECORD / RESULT MANAGEMENT

The **Academic Record Module** is responsible for storing and managing students' academic information.

It maintains details such as:

- Subject Code
- Subject Name
- Credits
- Marks
- Semester
- Grade
- Grade Point

Academic records are stored in the **MySQL database** and are associated with a particular student using the student's Roll Number. The module calculates the student's **grade and grade point** based on the marks. It also calculates the **average marks and overall CGPA** using subject credits.

The module is implemented using the `Subject`, `AcademicRecord`, and `AcademicDAO` classes.

Main functions: `addAcademicRecord()`, `getRecords()`, `calculateGrade()`, `getGradePoint()`, and result calculation.

Java concepts used: Classes, Objects, Encapsulation, Composition, JDBC, SQL, Exception Handling, and Calculations.

MODULE 4: ATTENDANCE MANAGEMENT

The **Attendance Management Module** is responsible for recording and displaying student attendance for different subjects.

It stores information such as:

- Subject Code
- Total Classes
- Attended Classes
- Attendance Percentage
- Attendance Status

The module calculates the attendance percentage using the total number of classes and attended classes.

The system also determines the attendance status. If the attendance percentage is **75% or above**, the student is shown as **Eligible**; otherwise, the status is shown as **Shortage**.

Attendance information is stored in the **MySQL database**.

The module is implemented using the `Attendance` and `AttendanceDAO` classes.

Main functions: `addAttendance()`, `getAttendance()`, `calculatePercentage()`, and `getStatus()`.

Java concepts used: Classes, Objects, Encapsulation, JDBC, MySQL, Validation, Exception Handling, and Mathematical Calculations.

MODULE 5: DATABASE AND JDBC MANAGEMENT

The **Database and JDBC Management Module** is responsible for connecting the Java application with the MySQL database and performing database operations.

The system uses **JDBC (Java Database Connectivity)** to communicate with MySQL.

The database contains tables for:

- Admins
- Students
- Subjects
- Academic Records
- Attendance

The `DBConnection` class manages the database connection, while the DAO classes perform operations such as **insert, search, update, delete, and retrieve**.

`PreparedStatement` is used for executing SQL queries and handling user-provided values.

The `DatabaseInitializer` class helps initialize the required database structure.

Main functions: Database connection, SQL execution, record insertion, record retrieval, record updating, and record deletion.

Java concepts used: JDBC, MySQL, SQL, `Connection`, `PreparedStatement`, `ResultSet`, Transactions, and Exception Handling.

MODULE 6: STUDENT ACADEMIC PORTAL / OOP MANAGEMENT

The **Student Academic Portal Module** integrates all the different modules into one application.

The `AcademicPortal` class provides the main application workflow and contains separate menus for **Admin and Student users**.

The system provides:

Admin Dashboard

- Add Student
- Update Student
- Delete Student
- Search Student
- View All Students
- Add/Update Marks
- View Student Result
- Add/Update Attendance
- View Attendance
- View Admin Profile

Student Dashboard

- View Student Profile
- View Academic Result
- View Attendance
- Logout

The project follows important **Object-Oriented Programming principles**. Classes such as `Person`, `Student`, `Admin`, `Subject`, `AcademicRecord`, and `Attendance` represent different entities in the system.

The `Student` and `Admin` classes inherit common information from the abstract `Person` class. Composition is also used, for example, an `AcademicRecord` contains a `Subject` object.

Main functions: `start()`, `adminLogin()`, `studentLogin()`, `adminMenu()`, `studentMenu()`, `displayResult()`, and `displayDetails()`.

Java concepts used: OOP, Classes, Objects, Encapsulation, Inheritance, Abstraction, Composition, Constructors, Methods, JDBC, MySQL, and Exception Handling.

Implementation

The **Student Academic Portal** is implemented using **Java, Object-Oriented Programming (OOP), JDBC, and MySQL**. The project is divided into modules for student management, administrator management, academic records, attendance, database connectivity, and the main portal.

The important implementation techniques used are **Encapsulation, Inheritance, Abstraction, Composition, JDBC database connectivity, CRUD operations, PreparedStatement, SQL queries, validation, transaction management, and result/attendance calculations**.

Module 1: STUDENT MANAGEMENT

PROGRAM:

```
public class Student extends Person {

    private String rollNo;
    private String branch;
    private int year;
    private String password;

    // Constructor
    public Student(String rollNo, String name, String phone,
                  String email, String branch,
                  int year, String password) {

        super(name, phone, email);

        this.rollNo = rollNo;
        this.branch = branch;
        this.year = year;
        this.password = password;
    }

    // Get student information
    public String getRollNo() {
        return rollNo;
    }

    public String getBranch() {
        return branch;
    }

    public int getYear() {
        return year;
    }

    public String getPassword() {
        return password;
    }

    // Display student details
    @Override
    public void displayDetails() {
        System.out.println("\n===== STUDENT PROFILE =====");
        System.out.println("Roll Number : " + rollNo);
        System.out.println("Name : " + getName());
        System.out.println("Phone : " + getPhone());
        System.out.println("Email : " + getEmail());
        System.out.println("Branch : " + branch);
        System.out.println("Year : " + year);
    }
}
```

```
}
```

Student Database Operations

The `StudentDAO` class performs database operations using JDBC.

```
public boolean addStudent(Student s) {

    String sql = ""
        INSERT INTO students
        (roll_no, name, phone, email, branch, year, password)
        VALUES (?, ?, ?, ?, ?, ?, ?)
        "";

    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, s.getRollNo());
        ps.setString(2, s.getName());
        ps.setString(3, s.getPhone());
        ps.setString(4, s.getEmail());
        ps.setString(5, s.getBranch());
        ps.setInt(6, s.getYear());
        ps.setString(7, s.getPassword());

        return ps.executeUpdate() == 1;

    } catch (SQLException e) {

        System.out.println("Add student error: "
            + e.getMessage());

    }

    return false;
}
```

Student Search Algorithm

1. Accept the student's Roll Number.
2. Connect to the MySQL database.
3. Execute SELECT query using Roll Number.
4. If a record is found, create a Student object.
5. Display the student details.
6. If no record is found, display "Student not found."

Main Functions:

- `addStudent()`
- `findStudent()`
- `updateStudent()`
- `deleteStudent()`
- `getAllStudents()`
- `loginStudent()`

Java concepts used: Classes, Objects, Inheritance, Encapsulation, JDBC, MySQL, CRUD operations, PreparedStatement, and ResultSet.

Module 2: ADMIN MANAGEMENT

The **Admin Management Module** manages administrator login and administrator information.

PROGRAM:

```
public class Admin extends Person {

    private int adminId;
    private String username;

    // Constructor
    public Admin(int adminId, String username,
                String name, String phone, String email) {

        super(name, phone, email);

        this.adminId = adminId;
        this.username = username;
    }

    public int getAdminId() {
        return adminId;
    }

    public String getUsername() {
        return username;
    }

    // Display admin details
    @Override
    public void displayDetails() {

        System.out.println("Admin ID : " + adminId);
        System.out.println("Username : " + username);
        System.out.println("Name      : " + getName());
        System.out.println("Phone    : " + getPhone());
        System.out.println("Email   : " + getEmail());
    }
}
```

Admin Login

```
public Admin login(String username, String password) {

    String sql = ""
        SELECT admin_id, username, name, phone, email
        FROM admins
        WHERE username = ? AND password = ?
        "";

    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, username);
        ps.setString(2, password);

        try (ResultSet rs = ps.executeQuery()) {

            if (rs.next()) {

                return new Admin(
                    rs.getInt("admin_id"),
                    rs.getString("username"),
```

```

        rs.getString("name"),
        rs.getString("phone"),
        rs.getString("email")
    );
    }
}

} catch (SQLException e) {

    System.out.println(
        "Admin login error: " + e.getMessage()
    );
}

return null;
}

```

Login Algorithm

1. Accept username and password.
2. Connect to MySQL.
3. Search the admins table.
4. Compare username and password.
5. If matching record exists, login is successful.
6. Otherwise, display invalid credentials.

Main function: login()

Java concepts used: Inheritance, Encapsulation, JDBC, PreparedStatement, SQL, Authentication, Classes, and Objects.

Module 3: ACADEMIC RECORD / RESULT MANAGEMENT

The **Academic Record Module** manages subjects, marks, semesters, grades, and grade points.

PROGRAM:

```

public class AcademicRecord {

    private Subject subject;
    private double marks;
    private int semester;

    // Constructor
    public AcademicRecord(Subject subject,
                          double marks,
                          int semester) {

        if (marks < 0 || marks > 100)
            throw new IllegalArgumentException(
                "Marks must be between 0 and 100."
            );

        if (semester < 1 || semester > 8)
            throw new IllegalArgumentException(
                "Semester must be between 1 and 8."
            );

        this.subject = subject;
        this.marks = marks;
        this.semester = semester;
    }

    public Subject getSubject() {

```

```

        return subject;
    }

    public double getMarks() {
        return marks;
    }

    public int getSemester() {
        return semester;
    }

    // Calculate grade
    public String calculateGrade() {

        if (marks >= 90) return "A+";
        if (marks >= 80) return "A";
        if (marks >= 70) return "B";
        if (marks >= 60) return "C";
        if (marks >= 50) return "D";

        return "F";
    }

    // Calculate grade point
    public double getGradePoint() {

        if (marks >= 90) return 10;
        if (marks >= 80) return 9;
        if (marks >= 70) return 8;
        if (marks >= 60) return 7;
        if (marks >= 50) return 6;

        return 0;
    }
}

```

Grade Calculation Algorithm

1. Read student marks.
2. Check whether marks are between 0 and 100.
3. Compare marks with grade ranges.
4. Assign the corresponding grade.
5. Assign the corresponding grade point.
6. Store the academic record in MySQL.

Academic Record Database Operation

```

public List<AcademicRecord> getRecords(String rollNo) {

    List<AcademicRecord> records = new ArrayList<>();

    String sql = ""
        SELECT s.subject_code, s.subject_name,
               s.credits, a.marks, a.semester
        FROM academic_records a
        JOIN subjects s
        ON a.subject_code = s.subject_code
        WHERE a.roll_no = ?
        ORDER BY a.semester, s.subject_code
        """;

    try (Connection con = DBConnection.getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setString(1, rollNo);

        try (ResultSet rs = ps.executeQuery()) {

            while (rs.next()) {

                Subject subject = new Subject(

```

```

        rs.getString("subject_code"),
        rs.getString("subject_name"),
        rs.getInt("credits")
    );

    records.add(
        new AcademicRecord(
            subject,
            rs.getDouble("marks"),
            rs.getInt("semester")
        )
    );
}

} catch (SQLException e) {

    System.out.println(
        "Read academic records error: "
        + e.getMessage()
    );
}

return records;
}

```

Main functions:

- addAcademicRecord()
- getRecords()
- calculateGrade()
- getGradePoint()

Java concepts used: Composition, Encapsulation, JDBC, SQL JOIN, PreparedStatement, ResultSet, Validation, and Calculations.

Module 4: ATTENDANCE MANAGEMENT

The **Attendance Management Module** records and calculates the attendance percentage of students for different subjects.

PROGRAM:

```

public class Attendance {

    private String subjectCode;
    private int totalClasses;
    private int attendedClasses;

    // Constructor
    public Attendance(String subjectCode,
                      int totalClasses,
                      int attendedClasses) {

        if (totalClasses <= 0)
            throw new IllegalArgumentException(
                "Total classes must be greater than zero."
            );

        if (attendedClasses < 0 ||
            attendedClasses > totalClasses)

            throw new IllegalArgumentException(
                "Invalid attended classes."
            );
    }
}

```

```

        );

        this.subjectCode = subjectCode;
        this.totalClasses = totalClasses;
        this.attendedClasses = attendedClasses;
    }

    // Calculate attendance percentage
    public double calculatePercentage() {

        return ((double) attendedClasses
                / totalClasses) * 100;
    }

    // Check attendance status
    public String getStatus() {

        return calculatePercentage() >= 75
            ? "Eligible"
            : "Shortage";
    }
}

```

Attendance Algorithm

1. Read total classes.
2. Read attended classes.
3. Validate the attendance values.
4. Calculate:

$$\text{Attendance \%} = (\text{Attended Classes} / \text{Total Classes}) \times 100$$

5. If attendance is 75% or more,
status = Eligible.
6. Otherwise,
status = Shortage.
7. Store the attendance record in MySQL.

Example

Total Classes = 40
Attended Classes = 32

Attendance =
(32 / 40) × 100

Attendance = 80%

Status = Eligible

Main functions:

- addAttendance()
- getAttendance()
- calculatePercentage()
- getStatus()

Java concepts used: Classes, Objects, Encapsulation, Validation, JDBC, MySQL, Mathematical Calculations, and Exception Handling.

Module 5: DATABASE AND JDBC MANAGEMENT

The **Database and JDBC Module** connects the Java application with the MySQL database.

The project uses JDBC classes such as:

- Connection
- DriverManager
- PreparedStatement
- Statement
- ResultSet

PROGRAM:

```
public class DBConnection {

    private static final String URL =
        "jdbc:mysql://localhost:3306/"
        + "student_academic_portal"
        + "?useSSL=false"
        + "&serverTimezone=Asia/Kolkata";

    private static final String USER = "root";

    private DBConnection() {}

    public static Connection getConnection()
        throws SQLException {

        return DriverManager.getConnection(
            URL,
            USER,
            DBConnectionPassword.get()
        );
    }
}
```

Database Initialization

```
public static void initialize() {

    createDatabase();
    createTables();
    createDefaultAdmin();
}
```

The system automatically checks whether the database exists and creates the required tables.

The main tables are:

```
admins
students
subjects
academic_records
attendance
```

Database Implementation Algorithm

1. Start the application.
2. Establish connection with MySQL.
3. Create student_academic_portal database if required.
4. Create required tables.

5. Create default admin if required.
6. Perform CRUD operations using DAO classes.
7. Close database resources using try-with-resources.

JDBC Technique Used

The project uses **PreparedStatement** instead of directly constructing SQL queries with user input.

Example:

```
PreparedStatement ps =
    con.prepareStatement(
        "SELECT * FROM students WHERE roll_no = ?"
    );

ps.setString(1, rollNo);

ResultSet rs = ps.executeQuery();
```

This makes database operations structured and helps prevent SQL injection.

Java concepts used: JDBC, MySQL, SQL, Connection, PreparedStatement, ResultSet, CRUD operations, Transactions, and Exception Handling.

Module 6: OOP MANAGEMENT / MAIN PROGRAM

The **OOP Management Module** combines all the modules and provides the main interface of the Student Academic Portal.

PROGRAM:

```
public class Main {

    public static void main(String[] args) {

        new AcademicPortal().start();

    }

}
```

The **AcademicPortal** class manages the complete application workflow.

Main Portal

```
public void start() {

    DatabaseInitializer.initialize();

    int choice;

    do {

        System.out.println(
            "\n=====
        );

        System.out.println(
            "          STUDENT ACADEMIC PORTAL"
        );

        System.out.println(
            "=====
        );

    }
```

```

        System.out.println("1. Admin Login");
        System.out.println("2. Student Login");
        System.out.println("0. Exit");

        choice = readInt("Enter your choice: ");

        switch (choice) {

            case 1 -> adminLogin();

            case 2 -> studentLogin();

            case 0 ->
                System.out.println(
                    "Thank you for using the portal!"
                );

            default ->
                System.out.println(
                    "Invalid choice."
                );

        }

    } while (choice != 0);
}

```

Student Login

```

private void studentLogin() {

    System.out.println(
        "\n===== STUDENT LOGIN ====="
    );

    String rollNo = read("Roll Number: ");
    String password = read("Password: ");

    Student student =
        studentDAO.loginStudent(
            rollNo,
            password
        );

    if (student != null) {

        System.out.println(
            "Login successful."
        );

        studentMenu(student);

    } else {

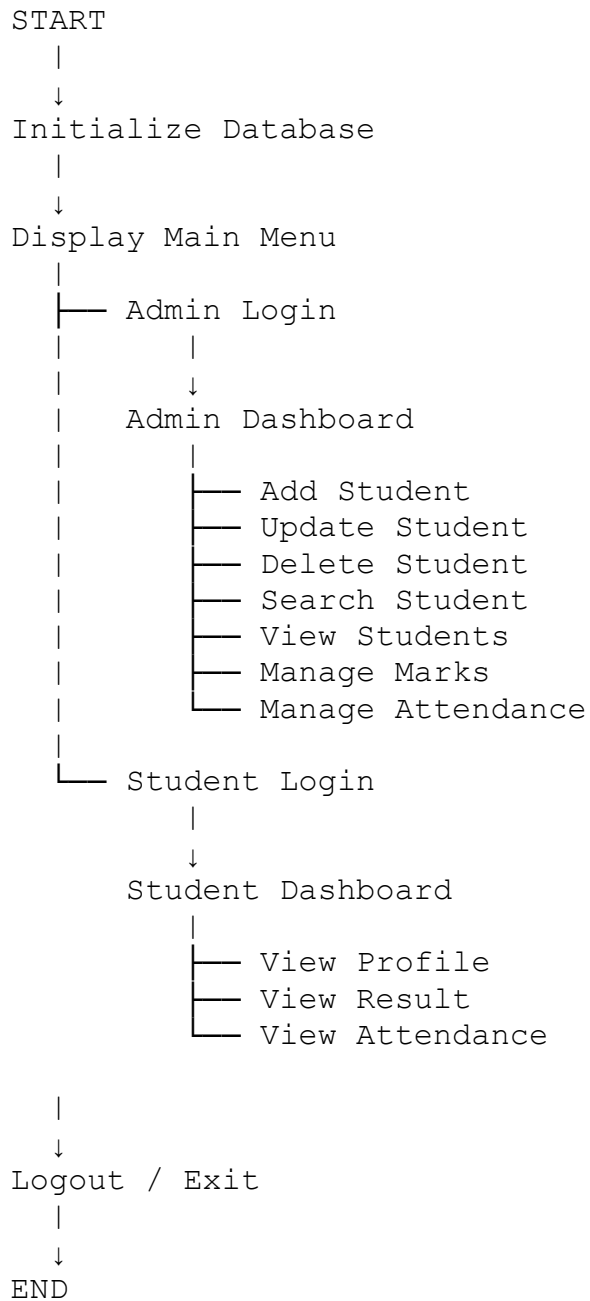
        System.out.println(
            "Invalid student credentials."
        );

    }

}

```

Overall Program Algorithm



OOP Concepts Used

Results

Figure 1:

```
=====
                        STUDENT ACADEMIC PORTAL
=====
1. Admin Login
2. Student Login
0. Exit
=====
Enter your choice: 1

===== ADMIN LOGIN =====
Username: admin
Password: admin123
Login successful.
```

Figure 2:

```
=====
                        ADMIN DASHBOARD
=====
1. Add Student
2. Update Student
3. Delete Student
4. Search Student
5. View All Students
6. Add/Update Marks
7. View Student Result
8. Add/Update Attendance
9. View Attendance
10. Admin Profile
0. Logout
Enter choice: 10

===== ADMIN PROFILE =====
Name      : System Administrator
Admin ID  : 1
Username  : admin
Phone     : 9876543210
Email     : admin@bvr.it.edu
```

Figure 3:

```
===== STUDENT DASHBOARD =====
Welcome, rithvik
2. View Academic Result
3. View Attendance
0. Logout
Enter choice: 1

===== STUDENT PROFILE =====
Roll Number : 577
Name        : rithvik
Phone       : 9080706050
Email       : 577@gmail.com
Branch      : cse
Year        : 2
```

Conclusion

The project “**Inventory Management System**” was successfully developed to address the efficient management and tracking of items, stock levels, suppliers, reorder requirements, and purchase orders. The proposed system provides an organized way to store and manage inventory information, identify low-stock items, process reorder requests, manage supplier details, and schedule purchase orders. It demonstrates the application of **Java data structures such as Hashtable and Queue, Enumeration, Calendar, static variables, and Object-Oriented Programming concepts**. The project helped in understanding how different Java concepts can be combined to develop a practical and efficient inventory management system.

Team Members

S.No.	Name	Roll No.
1	B.Rithvik	25211A0577
2	B.Harsha Vardhan	25211A0578
3	C.Mayank Sai	25211A05B1
4	D.Ravi Shankar	25211A05D0